
```
function [ISE, ISC] = func(tauI, color, label)
% This calculates ISE and ISC with a step change in the disturbance
% and plots the results with the specified color and label.

% Print the tauI value before each function call
fprintf('\n=== Results for tauI = %.2f ===\n', tauI);

% Initialize a figure
figure(1);
% Set up plotting layout
subplot(1, 2, 1); % y plot
subplot(1, 2, 2); % u plot

for i = [-1, 0, 1, 2]

    % Define constants
    tau1 = 3;
    tau2 = 2;
    tau3 = 1;
    k = 1;
    Kc = 10^i;

    % Define the transfer functions
    s = tf('s');
    Gp = Kc * (1 + 1/(tauI * s));
    Gm = k * (tau3*s + 1) / ((tau2*s + 1) * (tau1*s + 1));
    Gol = Gp * Gm;
    t = 0:0.01:50;

    % Compute system responses
    Y_over_Ysp = Gol/(1+Gol);
    y = step(Y_over_Ysp, t);
    U_over_Ysp = Gp * (1 - Gol/(1+Gol));
    u = step(U_over_Ysp, t);

    % Plot results with the specified color
    subplot(1, 2, 1);
    h1 = plot(t, y, 'Color', color); % Plot y with the specified color
    hold on;
    plot(t, zeros(1, length(t)), 'k--'); % Reference line for y
    xlabel('t');
    ylabel('y');

    subplot(1, 2, 2);
    h2 = plot(t, u, 'Color', color); % Plot u with the specified color
    hold on;
    plot(t, -1*ones(1, length(t)), 'k--'); % Reference line for u
    xlabel('t');
    ylabel('u');

    % Calculate ISE and ISC
    ISE = trapz(t, y.^2);
```

```

ISC = trapz(t, (u - (-1)).^2);

fprintf('Kc = 10^%d -> ISE: %.4f, ISC: %.4f\n', i, ISE, ISC);

end

% Add the legend with the label passed in
subplot(1, 2, 1);
legend({'System Response', 'Reference Line'}, 'Location', 'best');

subplot(1, 2, 2);
legend({'System Response', 'Reference Line'}, 'Location', 'best');
end

func(0.1, [1, 0, 0]);
func(1, [0, 1, 0]);
func(10, [0, 0, 1]);
func(100, [1, 0, 1]);

function plot_ise_isc_vs_kc_continuous()
    % Define continuous range of Kc using logspace for better resolution
    Kc_values = logspace(-2, 2, 200); % Avoid Kc = 0 for stability, covers
0.01 to 100
    tauI_values = [3, 1, 10, 100]; % Different tauI values

    % Create figure
    figure;
    hold on;

    % Loop through different TauI values
    for ti = 1:length(tauI_values)
        tauI = tauI_values(ti);
        ISE_ISC_sum = nan(size(Kc_values)); % Store (ISE + ISC), using NaN to
filter issues

        % Compute ISE + ISC for each Kc
        for ki = 1:length(Kc_values)
            Kc = Kc_values(ki);
            [ISE, ISC, isStable] = compute_ise_isc(tauI, Kc);
            if isStable
                ISE_ISC_sum(ki) = ISE + ISC;
            end
        end

        % Plot continuous curve
        plot(Kc_values, ISE_ISC_sum, 'DisplayName', sprintf('\tau_I = %.1f',
tauI), 'LineWidth', 2);
    end

    % Formatting
    set(gca, 'XScale', 'log', 'YScale', 'log'); % Log-log scale
    xlabel('Controller Gain (K_c)');
    ylabel('ISE + ISC');
    title('Performance Index (ISE + ISC) vs. Controller Gain (K_c)');

```

```

    legend('show', 'Location', 'best');
    grid on;
    hold off;
end

function [ISE, ISC, isStable] = compute_ise_isc(tauI, Kc)
    % Define system parameters
    tau1 = 3;
    tau2 = 2;
    tau3 = 1;
    k = 1;

    % Define the transfer functions
    s = tf('s');
    Gp = Kc * (1 + 1/(tauI * s));
    Gm = k * (tau3*s + 1) / ((tau2*s + 1) * (tau1*s + 1));
    Gol = Gp * Gm;
    t = 0:0.05:50; % Reduce resolution to speed up computation

    % Check system stability
    poles = pole(Gol/(1+Gol)); % Get closed-loop poles
    if any(real(poles) > 0) % Unstable system
        ISE = NaN;
        ISC = NaN;
        isStable = false;
        return;
    end
    isStable = true;

    % Compute system responses
    Y_over_Ysp = Gol/(1+Gol);
    y = step(Y_over_Ysp, t);
    U_over_Ysp = Gp * (1 - Gol/(1+Gol));
    u = step(U_over_Ysp, t);

    % Calculate ISE and ISC
    ISE = trapz(t, y.^2);
    ISC = trapz(t, (u - (-1)).^2);
end

% Run the function
plot_ise_isc_vs_kc_continuous();

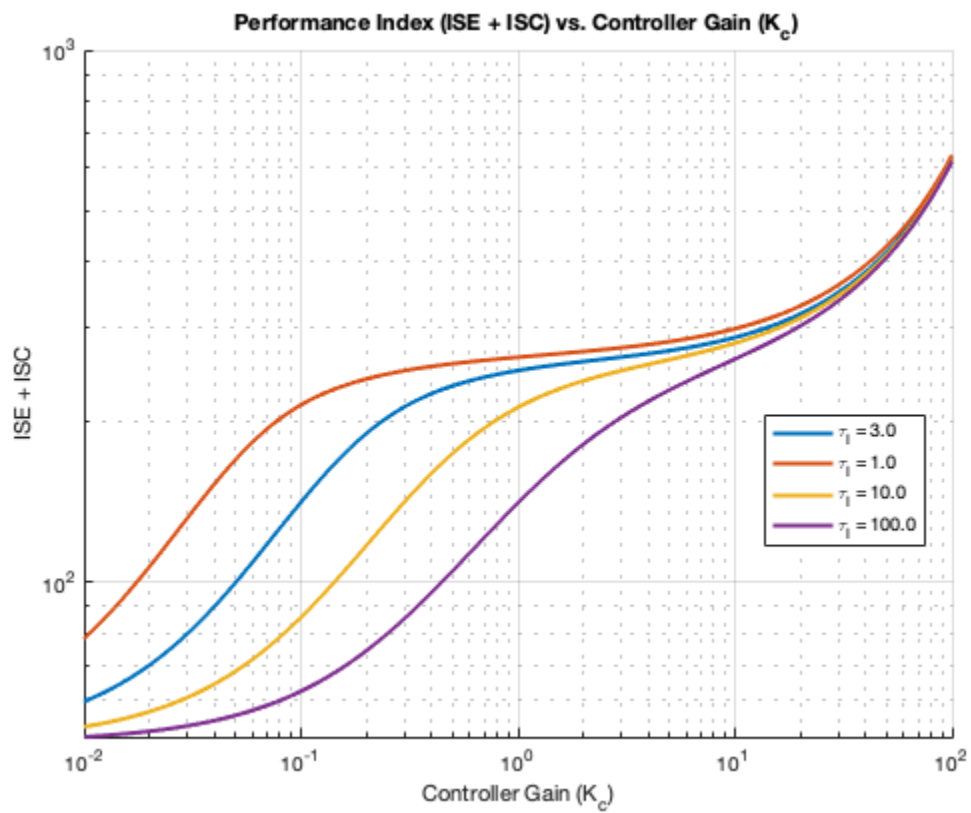
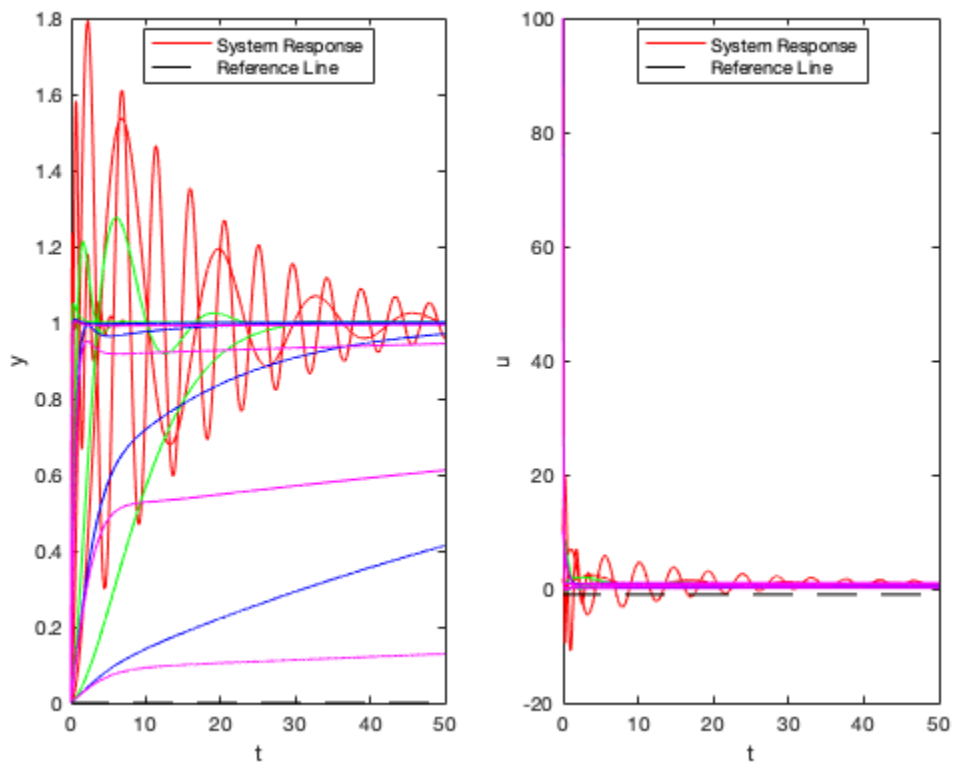
=== Results for tauI = 0.10 ===

=== Results for tauI = 1.00 ===

=== Results for tauI = 10.00 ===

=== Results for tauI = 100.00 ===

```



Published with MATLAB® R2024b